

异常报备上传bug修复

🔍 | 📖 正文 1.1 万字（6.4 页） | 浏览 22 次（1 人） | 查看全部 >

📄 AI 速览

异常报备流程优化决策项：

- 报备场景分类：**需确认是否按一级/二级分类展示（如客户问题-客户不在家），并明确哪些场景需取消预约单（如服务面积超限、客户表示不需要等）。
- 报备校验规则：**
 - 签到校验：**对“客户不在家”等场景，需强制劳动者签到后才能提交报备。
 - 证据要求：**分场景设定图片/视频或文字为必需项（如安全隐患仅需文字，物损需图片+文字）。
- 预约单取消流程：**
 - 自动取消条件：**命中特定场景（如客户下错地址）后，系统需实时取消订单并触发用户短信通知。
 - 取消原因记录：**标记为“劳动者端取消+报备分类”，供后续数据追溯。
- 客户不在家特殊处理：**
 - 短信触发：**系统自动发送短信，保留订单至预约时间+30分钟或触发后10分钟。
 - 劳动者操作：**需设计倒计时页面，支持“开始服务”或超时后“取消订单”。
- 空跑补偿规则：**明确补偿条件（如报备时间在预约前1小时内且订单取消）。

技术冲突修复决策项：

- 接口分离：**需评估是否将异常报备与自检报备接口拆分，避免逻辑混淆。
- 状态缓存：**当劳动者退出异常报备页面后，需通过Redis缓存状态，防止后续自检报备误渲染异常流程。
- 代码兼容性：**确认现有重试机制（如Redis操作重试）是否覆盖新场景，避免因异常导致流程中断。

其他关键项：

- 判责工单生成：**服务超限场景需自动生成工单，并关联判责结果决定补偿。
- 站长触达：**高危场景（如安全隐患）需通过公众号实时通知站长跟进。
- 数据展示优化：**海星平台需新增“报备时间”字段，并按分类展示问题详情。

在异常报备流程中，哪些场景需要劳动者强制签到后才能提交报备？当劳动者选择‘客户不在家’报备场景后，系统会触发哪些后续处理流程？

🖌️ 风格切换 ▾

💬 继续提问



【PRD】服务人员异常报备流程优化

一、需求背景

服务人员履约过程中，存在例如联系不上用户等异常场景，当前处理流程是引导服务人员联系商服->商服与用户核实情况->商服撤销锁定券码，整体处理流程长、效率低，且出于业务侧成本考虑，今年计划暂时取消商服角色，待业务规模上涨后建设完整的商服体系；在此背景下，计划优化劳动者端异常报备流程，劳动者基于异常情况可发起报备并系统自动取消预约单，以简化流程，提升人效。

二、需求收益

- 对用户：**提升用户体验，减少用户侧在交付异常场景下的进线
- 对劳动者：**简化履约中交付异常流程，尽快释放劳动者库存，并基于具体case给到劳动者空跑补贴

三、需求详情

3.1 报备场景梳理

- 注：自检报备不在本需求改动范围内

	报备一级分类	报备二级分类	是否涉及取消预约单
1	客户问题	客户要求提前开始/结束	否
2		客户仅要求完成部分区域	否
3		服务面积/场景超限	是
4		客户不在家	是
5		客户表示不需要了	是
6		客户下错地址	是
7		服务前已有物损	否
8		客户家停水停电	是
9		客户家有安全隐患	是
10	系统问题	系统问题无法签到签出	否
11	我的问题	迟到	否
12		临时原因无法服务	否
13		服务产生物损	否

3.2 需求详情

设计稿：<https://ingee.meituan.com/#/detail/194647>

需求分类	需求详情		设计稿		
异常报备服务人员操作流程优化	1、报备常规流程说明				
	• 异常报备入口（不变）：接单宝小程序订单详情页左下角				
	<table><tr><th>流程</th><th>详细说明</th></tr><tr><td>选择报备场景</td><td> - 标题：请选择您要报备的问题（单选） - 报备场景选择：按3.1中描述，分一级和二级报备展示 - 排列顺序：按照3.1中排序，默认展开客户问题列表 - 报备校验： </td></tr></table>	流程		详细说明	选择报备场景
流程	详细说明				
选择报备场景	- 标题：请选择您要报备的问题（单选） - 报备场景选择：按3.1中描述，分一级和二级报备展示 - 排列顺序：按照3.1中排序，默认展开客户问题列表 - 报备校验：				

场景一

- 若报备问题选择：1) 客户不在家 2) 服务面积/场景超限 3) 客户家停水停电，需劳动者[已完成签到](#)后才能提交报备
- 交互：选中上述三个报备项目且阿姨没有进行签到时，跳出签到弹窗
 - 弹窗标题：请先完成签到
 - 文字说明：完成签到后，你可以继续进行异常报备
 - 按钮：已到达，立即签到（签到限制同正常签到流程）

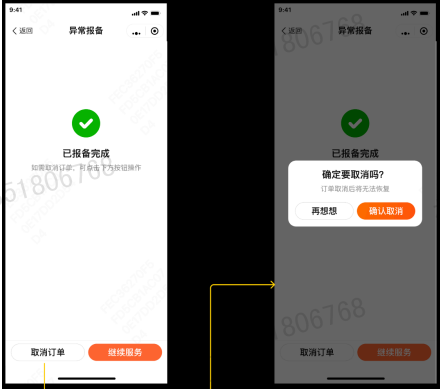
上传证据

- 包含模块：

模块	图示	详情
图片视频证据	02 请上传举证图片 查看拍摄示例 请拍摄能证明以上问题的图片(1-5张) 	■ 标题：请上传举证图片/视频 (最多5张) ■ 文字说明：最多可支持上传5张 ■ 限制：最多上传5张图片或视频 ■ 是否必须：见提交报备规则，若为必须项，在标题旁做提示
文字证据	03 请描述您具体情况 输入具体原因，不超过100字	■ 标题：请描述您的具体情况 ■ 文本框：输入具体原因，不得超过100字 ■ 是否必须：见提交报备规则，若为必须项，在标题旁做提示
提交按钮	提交 提交	■ 不满足提交报备规则时，按钮置灰，不支持点击

- 提交报备规则：
 - 分场景不同规则：

	报备一级分类	报备二级分类	所需证据
1	客户问题	客户要求提前开始/结束	仅文字为必需项
2		客户仅要求完成部分区域	仅文字为必需项
3		服务面积/场景超限	图片/视频+文字
4		客户不在家	图片/视频+文字



场景二



场景三



场景四



5		客户表示不需要了	仅文字为必需项
6		客户下错地址	仅文字为必需项
7		服务前已有物损	图片/视频+文字
8		客户家停水停电	图片/视频+文字
9		客户家有安全隐患	仅文字为必需项
10	系统问题	系统问题无法签到签出	图片/视频+文字
11	我的问题	迟到	仅文字为必需项
12		临时原因无法服务	仅文字为必需项
13		服务产生物损	图片/视频+文字

报备完成

- 【场景一】若命中以下报备场景，点击提交按钮后进入取消预约流程：

	报备一级分类	报备二级分类
1	客户问题	服务面积/场景超限
2		客户表示不需要了
3		客户下错地址
4		客户家停水停电
5		客户家有安全隐患

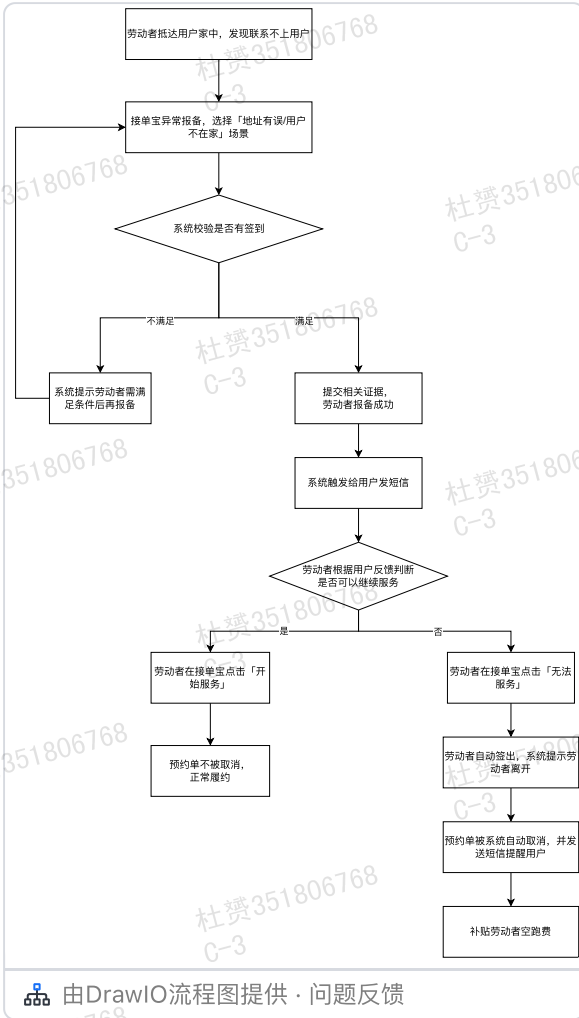
- 页面标题：已报备完成
 - 文字说明：如需取消订单，可点击下方按钮操作
 - 操作按钮：
 - 取消订单：点击后跳出二次确认弹窗
 - 弹窗标题：确定要取消吗？
 - 文字说明：订单取消后将无法恢复，请谨慎选择！
 - 按钮：再想想、确认取消，点击确认取消按钮后自动取消该笔预约单
 - 继续服务：点击后返回订单详情页
- 【场景二】若命中「迟到」和「临时原因无法服务」场景，点击提交按钮后进入报备完成页
 - 页面标题：已报备完成
 - 文字说明：请您主动联系用户说明情况
 - 按钮：联系用户说明情况，点击后返回订单详情页



	【场景三】若命中客户不在家场景，点击后流程在下方描述 【场景四】若命中其他场景，点击提交按钮后进入报备完成页 - 页面标题：已报备完成 - 按钮：返回订单详情页
--	---

2、【场景三】当选择「客户不在家」时，并点击提交按钮后的流程

- 整体流程图



- 具体流程说明

流程	详细说明
系统给用户发短信 该流程无需劳动者操作，系统自动触发 id4 p	- 触发节点：当劳动者在「客户不在家」下，点击「提交」后自动触发 - 短信发送对象：用户下单手机号 - 短信内容：您预约的今日\$点（预约开始时间）的美团无忧保洁服务，由于预约人不在家，系统将对订单保留到x点x分，请您尽快联系阿姨，超时将为您取消预约。 - 若触发时间在预约开始时间+30min内：预约单取消时间（保留时间）= 预约时间+30min； - 若触发事件在预约开始时间+30min后：预约单时间取消时间（保留时间）= 短信触发时间+10min

	服务人员确认状态 当劳动者提交异常报备后，进入该页面 页面标题：请稍等，已短信联系用户 文字说明：若取消时间倒计时仍未与用户取得联系，可点击左下方按钮取消订单；若联系上用户可正常开始服务，请点击右下方按钮开始服务 操作按钮： 开始服务：点击后自动返回订单详情页 取消订单： 按钮旁展示倒计时 点击条件：在x点x分*（见上）前置灰，不支持点击 二次确认弹窗： 弹窗标题：确定要取消吗？ 文字说明：订单取消后将无法恢复，请谨慎选择！ 选项：确认提交、再想想 状态变更：点击「取消订单」后，系统自动取消订单，无质检流程 若用户过程中退出异常报备页面，重新进入后直接进入该页面																													
	预约单取消流程 • 预约单取消说明： <table><tr><th></th><th>报备一级分类</th><th>报备二级分类</th><th>触发条件</th><th>触发短信</th></tr><tr><td>1</td><td rowspan="4">客户原因</td><td>客户表示不需要了</td><td>点击「取消订单」后实时取消该笔预约单，并实时触发短信</td><td>您今日\$点（预约开始时间）的美团无忧保洁服务，由于劳动者反馈您无需服务，系统已为您取消预约单，如有疑问请联系在线客服。</td></tr><tr><td>2</td><td>客户不在家4p</td><td>点击「取消订单」后实时取消该笔预约单，并实时触发短信</td><td>您今日\$点（预约开始时间）的美团无忧保洁服务，由于预约人不在家服务无法正常进行，系统已为您取消预约单，如有疑问请联系在线客服。</td></tr><tr><td>3</td><td>服务面积/场景超限</td><td>点击「取消订单」后实时取消该笔预约单，并实时触发短信</td><td>您今日\$点（预约开始时间）的美团无忧保洁服务，由于超出服务项范围服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。</td></tr><tr><td>4</td><td>客户下错地址</td><td>点击「取消订单」后实时取消该笔预约单，并实时触发短信</td><td>您今日\$点（预约开始时间）的美团无忧保洁服务，由于您预约地址有误服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>					报备一级分类	报备二级分类	触发条件	触发短信	1	客户原因	客户表示不需要了	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于劳动者反馈您无需服务，系统已为您取消预约单，如有疑问请联系在线客服。	2	客户不在家4p	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于预约人不在家服务无法正常进行，系统已为您取消预约单，如有疑问请联系在线客服。	3	服务面积/场景超限	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于超出服务项范围服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。	4	客户下错地址	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于您预约地址有误服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。				
	报备一级分类	报备二级分类	触发条件	触发短信																										
1	客户原因	客户表示不需要了	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于劳动者反馈您无需服务，系统已为您取消预约单，如有疑问请联系在线客服。																										
2		客户不在家4p	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于预约人不在家服务无法正常进行，系统已为您取消预约单，如有疑问请联系在线客服。																										
3		服务面积/场景超限	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于超出服务项范围服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。																										
4		客户下错地址	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于您预约地址有误服务无法进行，系统已为您取消预约单，如有疑问请联系在线客服。																										

		5	客户家有安全隐患	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于劳动者反馈服务异常，系统已为您取消预约单，如有疑问请联系在线客服。															
		6	客户家停水停电	点击「取消订单」后实时取消该笔预约单，并实时触发短信	您今日\$点（预约开始时间）的美团无忧保洁服务，由于客户侧停水停电服务无法正常进行，系统已为您取消预约单，如有疑问请联系在线客服。															
取消原因数据记录： ■ 标记为「劳动者端取消」+「报备一级分类-报备二级分类」，例如：劳动者端取消 客户问题-超出服务范围 ■ 展示模块：开店宝-交易后预约-查看详情-预约记录、海星平台-订单管理-订单详情-订单历史、太平洋																				
触达站长		触达方式：生活服务海星平台公众号 触达场景及内容： <table><tr><th></th><th>报备一级分类</th><th>报备二级分类</th><th>触达文案</th></tr><tr><td>1</td><td>客户问题</td><td>客户家有安全隐患 id9</td><td>您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 存在安全隐患无法正常服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$</td></tr><tr><td>2</td><td rowspan="2">我的问题</td><td>服务产生物损 id12</td><td>您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者服务产生物损，请及时跟进处理，具体报备内容见\$订单详情页链接\$</td></tr><tr><td>3</td><td>临时原因无法服务 id11</td><td>您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者原因无法服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$</td></tr></table>					报备一级分类	报备二级分类	触达文案	1	客户问题	客户家有安全隐患 id9	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 存在安全隐患无法正常服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$	2	我的问题	服务产生物损 id12	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者服务产生物损，请及时跟进处理，具体报备内容见\$订单详情页链接\$	3	临时原因无法服务 id11	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者原因无法服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$
	报备一级分类	报备二级分类	触达文案																	
1	客户问题	客户家有安全隐患 id9	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 存在安全隐患无法正常服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$																	
2	我的问题	服务产生物损 id12	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者服务产生物损，请及时跟进处理，具体报备内容见\$订单详情页链接\$																	
3		临时原因无法服务 id11	您的服务人员\$服务人员姓名\$报备预约单 \$预约单id（服务开始时间）\$ 因劳动者原因无法服务，请及时跟进处理，具体报备内容见\$订单详情页链接\$																	
自动生成判责工单		涉及报备场景及判责工单详情： <table><tr><th></th><th>报备一级分类</th><th>报备二级分类</th><th>生成判责单工单来源</th><th>问题场景</th></tr><tr><td>1</td><td>客户方问题</td><td>服务面积/场景超限 id3 p</td><td>劳动者报备（新增工单来源，手动发起工单时不支持手动选择）</td><td>服务人员拒单（新增问题场景）</td></tr></table> ■ 报备发起且订单被取消后实时生成判责工单					报备一级分类	报备二级分类	生成判责单工单来源	问题场景	1	客户方问题	服务面积/场景超限 id3 p	劳动者报备（新增工单来源，手动发起工单时不支持手动选择）	服务人员拒单（新增问题场景）					
	报备一级分类	报备二级分类	生成判责单工单来源	问题场景																
1	客户方问题	服务面积/场景超限 id3 p	劳动者报备（新增工单来源，手动发起工单时不支持手动选择）	服务人员拒单（新增问题场景）																

空跑劳动者补偿信号

- 涉及报备场景及规则：

	报备一级分类	报备二级分类	空跑补偿信号规则
1	客户方问题	客户不在家	该笔预约单劳动者报备该场景并取消订单
2		客户下错地址	该笔预约单劳动者报备该场景并取消订单，且报备时间在预约开始时间-1小时后
3		客户表示不需要了	该笔预约单劳动者报备该场景并取消订单，且报备时间在预约开始时间-1小时后
4		客户家停水停电	该笔预约单劳动者报备该场景并取消订单
5		客户家有安全隐患	该笔预约单劳动者报备该场景并取消订单
6		服务面积/场景超限	需同时满足： 1) 该笔预约单存在此报备并取消订单； 2) 自动生成的判责工单判责结果为「无责」

海星平台报备展示优化

- 报备问题字段展示「报备一级分类 - 报备二级分类」
- 新增字段：报备时间，记录提交报备的时间点
- 字段展示顺序：报备时间>报备问题>具体情况>报备图片
- 若选择「客户不在家」报备场景，展示「其他信息」字段
 - 字段逻辑：
 - 若服务人员选择「开始服务」，展示：于\$点\$分（点击开始服务按钮时间）正常开始服务；
 - 若服务人员选择「取消订单」，展示：于\$点\$分（点击无法服务按钮时间）确认无法服务并离开



异常报备和自检报备冲突问题

前异常报备和自检报备使用同一个方法进行处理，当用户在履约过程中，点击异常报备->选择客户不在家->退出->继续履约->自检报备->缺乏异常报备判断->渲染出异常报备信息，命中上述问题

异常报备和自检报备处理冲突问题：自营手艺人服务的异常报备和自检报备调用同一个接口，在手艺人履约过程中，如果客户不在家，手艺人异常报备选项为用户不在家，此时系统自动给客户发送短信通知并计时，超时则取消；联系上可以继续服务，此时如果手艺人没有点开始服务直接推出了异常报备的页面，在后续点击自检报备就会跳转异常报备的这个逻辑，导致了调用逻辑混乱。reportExceptionTemplate

代码块

```
1 package com.dianping.beauty.technician.action.doorToDoor;
2
3 import com.alibaba.fastjson.JSON;
4 import com.amazonaws.ClientConfiguration;
5 import com.amazonaws.Protocol;
6 import com.amazonaws.auth.AWSCredentials;
```

```

1 import com.amazonaws.auth.BasicAWSCredentials;
2 import com.amazonaws.services.s3.AmazonS3;
3 import com.amazonaws.services.s3.AmazonS3Client;
4 import com.amazonaws.services.s3.S3ClientOptions;
5 import com.dianping.beauty.ibot.dto.TextBody;
6 import com.dianping.beauty.ibot.enums.Pusher;
7 import com.dianping.beauty.ibot.service.common.CommonPushBaseService;
8 import com.dianping.beauty.technician.action.doorToDoor.dto.*;
9 import com.dianping.beauty.technician.action.doorToDoor.enums.*;
10 import com.dianping.beauty.technician.action.doorToDoor.exception.DoorToDoorException;
11 import com.dianping.beauty.technician.action.doorToDoor.facade.request.SwarmCallBackRequest;
12 import com.dianping.beauty.technician.action.doorToDoor.utils.DoorToDoorUtils;
13 import com.dianping.beauty.technician.utils.LionConfigUtil;
14 import com.dianping.cat.Cat;
15 import com.dianping.lion.Environment;
16 import com.dianping.lion.client.Lion;
17 import com.dianping.lion.common.util.JsonUtils;
18 import com.dianping.open.platform.mvc.dispatch.UriMapping;
19 import com.dianping.open.platform.mvc.interceptor.WebContext;
20 import com.dianping.squirrel.client.StoreKey;
21 import com.dianping.squirrel.client.impl.redis.RedisStoreClient;
22 import com.dianping.technician.common.api.domain.TechnicianResp;
23 import com.dianping.technician.common.tools.exeception.BizException;
24 import com.dianping.technician.common.tools.utils.ResponseMaker;
25 import com.dianping.technician.dto.SMSRequest;
26 import com.github.rholder.retry.Retryer;
27 import com.github.rholder.retry.RetryerBuilder;
28 import com.github.rholder.retry.StopStrategies;
29 import com.github.rholder.retry.WaitStrategies;
30 import com.google.common.collect.Maps;
31 import com.meituan.image.client.ImageUploadClient;
32 import com.meituan.image.client.impl.ImageUploadClientImpl;
33 import com.meituan.image.client.pojo.ImageResult;
34 import com.sankuai.lifevent.serviceprovider.common.dto.Response;
35 import com.sankuai.lifevent.serviceprovider.domain.api.SpTechnicianDomainService;
36 import com.sankuai.lifevent.serviceprovider.domain.api.dto.QueryTechnicianAreaRelationReq;
37 import com.sankuai.lifevent.serviceprovider.domain.api.dto.SpTechnicianDTO;
38 import com.sankuai.lifevent.serviceprovider.domain.api.dto.SpTechnicianQueryRequest;
39 import com.sankuai.lifevent.serviceprovider.domain.api.dto.TechnicianAreaRelationDTO;
40 import com.sankuai.lifevent.serviceprovider.merchant.api.ServiceProviderQueryService;
41 import com.sankuai.lifevent.serviceprovider.merchant.api.dto.ServiceProviderDTO;
42 import com.sankuai.technician.api.doorToDoor.tech.dto.*;
43 import com.sankuai.technician.api.doorToDoor.tech.enums.DoorToDoorMediaStatusEnum;
44 import com.sankuai.technician.api.doorToDoor.tech.enums.DoorToDoorTakeOrderRecordStatus;
45 import com.sankuai.technician.api.doorToDoor.tech.enums.ReportTypeEnum;
46 import com.sankuai.technician.api.doorToDoor.tech.enums.ServiceStageEnum;

```

```

53 import com.sankuai.technician.api.doorToDoor.tech.service.*;
54 import com.sankuai.technician.api.doorToDoor.tech.service.request.TechnicianSelfTestRequest;
55 import com.sankuai.technician.api.doorToDoor.tech.service.request.TechnicianUpdateMediaRequest;
56 import com.sankuai.technician.trade.api.doortodoor.dto.WorkOrderProblemChoiceDT0;
57 import com.sankuai.technician.trade.api.doortodoor.dto.WorkOrderProblemDT0;
58 import com.sankuai.technician.trade.api.doortodoor.enums.AssessAttrKeyEnum;
59 import com.sankuai.technician.trade.api.doortodoor.request.CreateOrderAndAttrRequest;
60 import com.sankuai.technician.trade.api.doortodoor.service.AssessLiabilityOrderProcessService;
61 import com.sankuai.trade.general.reserve.enums.ReserveOperatorEnum;
62 import com.sankuai.trade.general.reserve.request.CancelOrderRequest;
63 import com.sankuai.trade.general.reserve.response.ReserveResponse;
64 import com.sankuai.trade.general.reserve.service.ReserveOrderProcessService;
65 import lombok.extern.slf4j.Slf4j;
66 import org.apache.commons.collections.MapUtils;
67 import org.apache.commons.collections4.CollectionUtils;
68 import org.apache.commons.compress.utils.Lists;
69 import org.apache.commons.lang3.StringUtils;
70 import org.apache.thrift.TException;
71 import org.springframework.beans.BeanUtils;
72 import org.springframework.beans.factory.annotation.Autowired;
73 import org.springframework.stereotype.Component;
74 import com.dianping.technician.service.TechnicianSendMessageService;
75 import org.springframework.transaction.annotation.Transactional;
76
77 import javax.annotation.Resource;
78 import javax.crypto.Mac;
79 import javax.crypto.SecretKey;
80 import javax.crypto.spec.SecretKeySpec;
81 import java.nio.charset.StandardCharsets;
82 import java.text.DateFormat;
83 import java.text.SimpleDateFormat;
84 import java.util.*;
85 import java.util.concurrent.TimeUnit;
86 import java.util.stream.Collectors;
87
88 import static com.sankuai.technician.api.doorToDoor.tech.enums.DoorToDoorCategoryEnum.*;
89
90 @Component
91 @RequestMapping(value = "/doortodoor/upload", interceptors = {"loginInterceptor", "cross"})
92 @Slf4j
93 public class DoorToDoorUploadAction extends DoorToDoorBaseAction {
94
95     @Autowired
96     private DoorToDoorTakeOrderRecordService doorToDoorTakeOrderRecordService;
97
98     @Autowired
99     private DoorToDoorMediaService doorToDoorMediaService;

```



```
99  @Resource
100  private DoorToDoorTechServiceOrderQueryService doorToDoorTechServiceOrderQueryService;
101  @Autowired
102  private DoorToDoorTechnicianReportExceptionService doorToDoorTechnicianReportExceptionService;
103  @Autowired
104  private DoorToDoorTechnicianSelfTestService doorToDoorTechnicianSelfTestService;
105  @Autowired
106  private DoorToDoorTaskCenterService doorToDoorTaskCenterService;
107  @Autowired
108  private DoorToDoorServiceBcpService doorToDoorServiceBcpService;
109  @Autowired
110  private TechnicianSendMessageService technicianSendMessageService;
111  @Autowired
112  private AssessLiabilityOrderProcessService assessLiabilityOrderProcessService;
113  @Autowired
114  private SpTechnicianDomainService spTechnicianDomainService;
115  @Autowired
116  private ServiceProviderQueryService serviceProviderQueryService;
117  @Autowired
118  private CommonPushBaseService commonPushBaseService;
119  @Autowired
120  private ReserveOrderProcessService reserveOrderProcessService;
121  @Resource(name = "redisClientMedical")
122  private RedisStoreClient redisStoreClient;
123  private final static String CATEGORY = "doortodoor_upload_action";
124  private static String CLIENT_ID = "technician-mobile-web.doortodoor.venus.client.id";
125  private static String CLIENT_SECRET = "technician-mobile-web.doortodoor.venus.client.secret";
126  private static String VENUS_BUCKET_NAME = "technicianfulfillment";
127  private static final int NEW_VERSION = 3;
128  // 自营保洁拍摄服务后视频的stage
129  private static final int CLEAN_SELF_OWN_BRAND_UPLOAD_VIDEOS_AFTER_SERVICE = 5;
130  private static ImageUploadClient client;
131  // 添加重试器
132  public static Retryer<String> retryer = RetryerBuilder.<String>newBuilder()
```



```
145     .retryIfException() // 发生异常时重试
146     .retryIfResult(Objects::isNull) // 结果为null时重试
147     .withWaitStrategy(WaitStrategies.fixedWait(100, TimeUnit.MILLISECONDS)) // 重试间隔100ms
148     .withStopStrategy(StopStrategies.stopAfterAttempt(3)) // 最多重试3次
149     .build();
150
151     // 添加重试器
152     public static Retryer<Date> dateRetryer = RetryerBuilder.<Date>newBuilder()
153         .retryIfException() // 发生异常时重试
154         .retryIfResult(Objects::isNull) // 结果为null时重试
155         .withWaitStrategy(WaitStrategies.fixedWait(100, TimeUnit.MILLISECONDS)) // 重试间隔100ms
156         .withStopStrategy(StopStrategies.stopAfterAttempt(3)) // 最多重试3次
157         .build();
158
159     // 在类的顶部添加专门用于 set 操作的重试器
160     public static Retryer<Void> setRetryer = RetryerBuilder.<Void>newBuilder()
161         .retryIfException() // 发生异常时重试
162         .withWaitStrategy(WaitStrategies.fixedWait(100, TimeUnit.MILLISECONDS)) // 重试间隔100ms
163         .withStopStrategy(StopStrategies.stopAfterAttempt(3)) // 最多重试3次
164         .build();
165
166     static {
167         client = new ImageUploadClientImpl(VENUS_BUCKET_NAME, Lion.getString(Environment.getAppName(), CLIENT_ID), Lion.getString(Environment.getAppName(),
CLIENT_SECRET));
168         client.setAutoSetEnv();
169     }
170
171     private static AmazonS3 s3client;
172
173     private static String ACCESS_KEY_LION = "technician-mobile-web.doortodoor.s3.access.key";
174     private static String SECRET_KEY_LION = "technician-mobile-web.doortodoor.s3.secret.key";
175     private static String ENDPOINT_LION = "technician-mobile-web.doortodoor.s3.endpoint";
176     private static String S3BUCKET_NAME = "lifeevent-tech-video";
177     private static final String MAC_NAME = "HmacSHA1";
178     private static final String ENCODING = "UTF-8";
179     private static final String UPLOAD_TEXT_LION = "technician-mobile-web.doortodoor.upload.text";
180     private static final Integer EXAMPLE_TYPE = 1;
181     private static final String INSPECTION_VERSION_KEY = "inspection.report.version";
182     private static final String CATEGORY_VALUE = "status";
183
184     //accessKey:用户的Access Key ID
185     //secretKey:用户的Access Key Secret
186     //hostname:MSS的endpoint服务地址
187     static {
188         AWSCredentials credentials = new BasicAWSCredentials(Lion.getString(Environment.getAppName(), ACCESS_KEY_LION), Lion.getString(Environment.getAppName(),
SECRET_KEY_LION));
189         ClientConfiguration configuration = new ClientConfiguration();
```

```

190 // 默认协议为HTTPS。将这个值设置为Protocol.HTTP, 则使用的是HTTP协议
191 configuration.setProtocol(Protocol.HTTPS);
192 //生成云存储api client
193 s3client = new AmazonS3Client(credentials, configuration);
194 // 配置云存储服务地址
195 // 注意Endpoint只填写域名, 不包含协议( http:// 或 https:// )
196 s3client.setEndpoint(ENDPOINT_LION);
197 // 设置客户端生成的http请求hos格式, 目前只支持path type的格式, 不支持bucket域名的格式
198 S3ClientOptions s3ClientOptions = new S3ClientOptions();
199 s3ClientOptions.setPathStyleAccess(true);
200 s3client.setS3ClientOptions(s3ClientOptions);
201 }
202
203 private byte[] hmacSHA1Encrypt(String encryptText, String encryptKey) throws Exception {
204     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.hmacSHA1Encrypt(java.lang.String,java.lang.String)");
205     byte[] data = encryptKey.getBytes(ENCODING);
206     // 根据给定的字节数组构造一个密钥, 第二参数指定一个密钥算法的名称
207     SecretKey secretKey = new SecretKeySpec(data, MAC_NAME);
208     // 生成一个指定 Mac 算法 的 Mac 对象
209     Mac mac = Mac.getInstance(MAC_NAME);
210     // 用给定密钥初始化 Mac 对象
211     mac.init(secretKey);
212     byte[] text = encryptText.getBytes(ENCODING);
213     // 完成 Mac 操作
214     return mac.doFinal(text);
215 }
216
217 @UriMapping(value = "/presignedpic")
218 public Object presignedPic() {
219     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.presignedPic()");
220     try {
221         int techId = getTechId();
222         if (techId <= 0) {
223             return TechnicianResp.fail("需要登录才能上传视频!");
224         }
225         String purpose = getStringParameter("purpose");
226         if (!"idcard".equals(purpose)) {
227             // 服务单id
228             String serviceOrderId = getStringParameter("serviceorderid");
229             // 校验服务单和手艺人是否匹配
230             TechnicianResp<DoorToDoorTakeOrderRecordDTO> technicianResp = doorToDoorTakeOrderRecordService.queryTakeOrderByTechIdAndOrderId(techId,
231 Long.parseLong(serviceOrderId));
232             if (technicianResp.respFail() || technicianResp.getData() == null) {
233                 return TechnicianResp.fail("手艺人与预约单状态不匹配");
234             }
235         }
236         ImageResult result = client.getToken2();

```

```

236         if (result.isSuccess()){
237             Map<String, Object> map = ResponseMaker.noRoot()
238                 .merge("code", 200)
239                 .merge("signature", result.getToken())
240                 .merge("expireTime", result.getExpireTime())
241                 .merge("bucket", VENUS_BUCKET_NAME)
242                 .result();
243             return TechnicianResp.success(map);
244         } else {
245             return TechnicianResp.fail("图片上传授权异常! ");
246         }
247     } catch (Exception e) {
248         log.error("获取图片上传授权异常", e);
249     }
250 }
251 return TechnicianResp.fail("图片上传授权异常! ");
252 }
253
254 /**
255  * 构造视频对象文件名, 为保证唯一性, 由userId、时间戳以及文件名拼接构成
256  *
257  * @param fileName
258  * @param userId
259  * @return
260 */
261 private String buildObjectName(String fileName, long userId) {
262     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.buildObjectName(java.lang.String,long)");
263     StringBuilder objectNameBuilder = new StringBuilder("");
264     objectNameBuilder.append(userId);
265     objectNameBuilder.append("_");
266     objectNameBuilder.append(System.currentTimeMillis());
267     objectNameBuilder.append("_");
268     objectNameBuilder.append(fileName);
269     String objectName = objectNameBuilder.toString();
270     return objectName;
271 }
272
273 @UriMapping(value = "/videosignature")
274 public Object presignedVideo() {
275     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.presignedVideo()");
276     int techId = getTechId();
277     if (techId <= 0) {
278         return TechnicianResp.fail("需要登录才能上传视频! ");
279     }
280
281     //服务单id
282     String serviceOrderId = getStringParameter("serviceorderid");

```

282 // 校验服务单和手艺人是否匹配
283 TechnicianResp<DoorToDoorTakeOrderRecordDTO> technicianResp = doorToDoorTakeOrderRecordService.queryTakeOrderByTechIdAndOrderId(techId,
Long.parseLong(serviceOrderId));
284 if (technicianResp.respFail() || technicianResp.getData() == null) {
285 return TechnicianResp.fail("手艺人与预约单状态不匹配");
286 }
287
288 String fileName = WebContext.getRequest().getParameter("filename");
289 String objectName = buildObjectName(fileName, techId);
290 String transObjectName = objectName.substring(0, objectName.lastIndexOf(".")) + ".mp4";
291 String videoUrl = "https://" + ENDPOINT_LION + "/" + S3BUCKET_NAME + "/" + objectName;
292 String transUrl = "https://" + ENDPOINT_LION + "/" + S3BUCKET_NAME + "/trans_" + transObjectName;
293 List<String> list = getSignature();
294 Map<String, Object> map = ResponseMaker.noRoot()
295 .merge("AWSAccessKeyId", Lion.getString(Environment.getAppName(), ACCESS_KEY_LION))
296 .merge("policy", list.get(1))
297 .merge("signature", list.get(0))
298 .merge("fileName", objectName)
299 .merge("videoUrl", videoUrl)
300 .merge("transUrl", transUrl)
301 .result();
302 return TechnicianResp.success(map);
303 }
304
305 @UriMapping(value = "/transcode/callback")
306 public Object transcodeCallback() {
307 Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.transcodeCallback()");
308 try {
309 SwarmCallBackRequest swarmCallBackRequest = getRequest(SwarmCallBackRequest.class);
310 if (swarmCallBackRequest == null || swarmCallBackRequest.getStatus() != 0 || StringUtils.isBlank(swarmCallBackRequest.getId())) {
311 log.error("调用转码服务失败, response={}", JSON.toJSONString(swarmCallBackRequest), new DoorToDoorException("调用转码服务失败"));
312 return TechnicianResp.fail(500, "fail");
313 }
314 log.info("swarm transcodeCallback request {}", JSON.toJSONString(swarmCallBackRequest));
315 String scope = swarmCallBackRequest.getScope();
316 String saveAs = swarmCallBackRequest.getSaveAs();
317 String videoName = scope.substring(scope.lastIndexOf("/") + 1);
318 String transName = saveAs.substring(scope.lastIndexOf("/") + 1);
319 String videoUrl = "https://" + Lion.getString(Environment.getAppName(), ENDPOINT_LION) + "/" + S3BUCKET_NAME + "/" + videoName;
320 String transUrl = "https://" + Lion.getString(Environment.getAppName(), ENDPOINT_LION) + "/" + S3BUCKET_NAME + "/" + transName;
321 TechnicianUpdateMediaRequest request = new TechnicianUpdateMediaRequest();
322 request.setOriginUrl(videoUrl);
323 request.setVideoTranscodeUrl(transUrl);
324 request.setStatus(DoorToDoorMediaStatusEnum.TRANSCODING_SUCCESS.getCode());
325 TechnicianResp res = doorToDoorMediaService.updateMedia(request);
326 if (res == null || res.getCode() != 200) {
log.error("视频转码失败", new DoorToDoorException("保存视频封面失败"));
}

```
328    }
329    DoorToDoorSubmitVideoAuditRequest auditRequest = new DoorToDoorSubmitVideoAuditRequest();
330    auditRequest.setOriginUrl(videoUrl);
331    doorToDoorMediaService.submitVideoAudit(auditRequest);
332    return TechnicianResp.success("保存成功", null);
333 } catch (Exception e) {
334     log.error("转码回调服务异常", e);
335 }
336 return TechnicianResp.fail(500, "fail");
337 }
338
339 @UriMapping(value = "/videoframe/callback")
340 public Object videoframeCallback() {
341     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.videoframeCallback()");
342     try {
343         // 请求
344         SwarmCallBackRequest swarmCallBackRequest = getRequest(SwarmCallBackRequest.class);
345         if (swarmCallBackRequest == null || swarmCallBackRequest.getStatus() != 0 || StringUtils.isBlank(swarmCallBackRequest.getId())) {
346             log.error("调用视频帧截图服务失败, response={}", JSON.toJSONString(swarmCallBackRequest), new DoorToDoorException("转码失败"));
347             return TechnicianResp.fail(500, "fail");
348         }
349         log.info("swarm videoframeCallback request {}", JSON.toJSONString(swarmCallBackRequest));
350         String scope = swarmCallBackRequest.getScope();
351         String saveAs = swarmCallBackRequest.getSaveAs();
352         String videoName = scope.substring(scope.lastIndexOf("/") + 1);
353         String picName = saveAs.substring(saveAs.lastIndexOf("/") + 1);
354         String videoUrl = "https://" + Lion.getString(Environment.getAppName(), ENDPOINT_LION) + "/" + S3BUCKET_NAME + "/" + videoName;
355         String picUrl = "https://" + Lion.getString(Environment.getAppName(), ENDPOINT_LION) + "/" + S3BUCKET_NAME + "/" + picName;
356         TechnicianUpdateMediaRequest request = new TechnicianUpdateMediaRequest();
357         request.setOriginUrl(videoUrl);
358         request.setVideoHeadPic(picUrl);
359         TechnicianResp res = doorToDoorMediaService.updateMedia(request);
360         if (res == null || res.getCode() != 200) {
361             log.error("保存视频封面图服务失败", new DoorToDoorException("保存视频封面失败"));
362         }
363         return TechnicianResp.success("保存成功", null);
364     } catch (Exception e) {
365         log.error("调用获取视频帧服务失败", e);
366     }
367     return TechnicianResp.fail(500, "fail");
368 }
369
370 public List<String> getSignature() {
371     Cat.logEvent("INVALID_METHOD", "com.dianping.beauty.technician.action.doorToDoor.DoorToDoorUploadAction.getSignature()");
372     String bucketName = S3BUCKET_NAME;
373     String accessKey = Lion.getString(Environment.getAppName(), ACCESS_KEY_LION);
```

374375376377378379380381382383384385386387388389390391392393394395396397398399400401402403404405406407408409410411412413414415416417418419

String secretKey = Lion.getString(Environment.getAppname(), SECRET_KEY_LION);String expiration;String signature;String policyEncoded;String policy;

// 将date转换为指定格式并设置过期时间为1天后 (这个过期时间可以根据自己业务需求自定, 应该在满足自己业务需求的前提下尽量小, 才更安全)Date date = new Date();DateFormat df = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss'Z'");Calendar calendar = new GregorianCalendar();calendar.setTime(date);calendar.add(Calendar.DATE, 1);date = calendar.getTime();df.setTimeZone(TimeZone.getTimeZone("GMT"));// 格式如"2018-12-01T00:00:00Z"expiration = df.format(date);policy = String.format("{\"expiration\":\"%s\", \"conditions\": [{\"bucket\":\"%s\"}, {\"starts-with\": \"%key\", \"%\"}]", expiration, bucketName);policyEncoded = Base64.getEncoder().encodeToString(policy.getBytes(StandardCharsets.UTF_8));try {signature = Base64.getEncoder().encodeToString(hmacSHA1Encrypt(policyEncoded, secretKey));} catch (Exception e) {log.error("getNormalUploadSign - 签名生成异常信息 {}, 异常堆栈信息 {}", e.getMessage(), e);throw new RuntimeException();}List<String> list = Lists.newArrayList();list.add(signature);list.add(policyEncoded);list.add(accessKey);return list;

@UriMapping(value = "/uploadtext")public Object uploadText() {Long serviceOrderId = getLongParameter("serviceOrderId");if (serviceOrderId <= 0) {return TechnicianResp.fail("参数错误");}TechnicianResp<TechServiceOrderDT0> serviceOrderResp = null;try {serviceOrderResp = doorToDoorTechServiceOrderQueryService.findDetailByOrderId(serviceOrderId);} catch (Exception e) {log.error("upload text query service order fail", e);}if (serviceOrderResp == null || serviceOrderResp.getData() == null || StringUtils.isEmpty(serviceOrderResp.getData().getExtraInfo())) {return TechnicianResp.fail("获取类目发生异常");}

```
String categoryId = DoorToDoorUtils.getServiceOrderCategory(serviceOrderResp.getData());
if (StringUtils.isEmpty(categoryId)) {
    return TechnicianResp.fail("获取的类目为空");
}
Integer serviceStage = getIntParameter("serviceStage");
if (serviceStage <= 0) {
    return TechnicianResp.fail("参数错误");
}
UploadTextInfo textInfo = new UploadTextInfo();
textInfo.setExampleText("查看拍摄示例");
textInfo.setExampleTitle("拍摄示例");
textInfo.setExampleButtonText("我知道了");
if (getVersion() >= NEW_VERSION && serviceStage == CLEAN_SELF_OWN_BRAND_UPLOAD_VIDEOS_AFTER_SERVICE) {
    textInfo.setUploadButtonText("上传视频并签出");
}else {
    textInfo.setUploadButtonText("上传");
}
List<UploadTextConfigItem> configItemList = null;
try {
    configItemList = Lion.getList(Environment.getAppName(), UPLOAD_TEXT_LION, UploadTextConfigItem.class);
} catch (Exception e) {
    log.error("获取lion配置项technician-mobile-web.doortodoor.upload.text发生异常", e);
}

if (CollectionUtils.isEmpty(configItemList)) {
    return TechnicianResp.success(textInfo);
}

List<UploadTextInfoItem> textInfoItemList = new ArrayList<>();
List<UploadTextExampleItem> exampleItemList = new ArrayList<>();
for (UploadTextConfigItem item : configItemList) {
    // 这里面是根据categoryId来获取对应行业的id
    // 家电数码维修和家庭维修使用相同的lion配置
    if (!categoryId.equals(APPLIANCE_REPAIR.getId())) {
        if (!categoryId.equals(item.getCategoryId()) || (item.getServiceStage() != null && !serviceStage.equals(item.getServiceStage()))){
            // serviceStage未配置不continue
            continue;
        }
    }else{
        // 说明是家电维修
        if (!DIGITAL_APPLIANCE_REPAIR.getId().equals(item.getCategoryId()) || (item.getServiceStage() != null && !serviceStage.equals(item.getServiceStage()))){
            continue;
        }
    }
}

if (EXAMPLE_TYPE.equals(item.getConfigType())) {
```



466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511

```
// 配置的是示例, 根据configType进行判断
UploadTextExampleItem exampleItem = new UploadTextExampleItem();
exampleItem.setTitle(item.getTitle());
exampleItem.setSubtitle(item.getSubtitle());
exampleItem.setTip(item.getTip());
exampleItem.setUrl(item.getUrl());
exampleItem.setUrls(item.getUrls());
exampleItem.setVideoCover(item.getVideoCover());
exampleItem.setType(item.getType());
exampleItemList.add(exampleItem);
} else {
    UploadTextInfoItem textInfoItem = new UploadTextInfoItem();
    textInfoItem.setId(item.getId());
    textInfoItem.setTitle(item.getTitle());
    if (CollectionUtils.isEmpty(item.getSubtitle())) {
        textInfoItem.setSubtitle(item.getSubtitle().get(0).getText());
    }
    textInfoItem.setMinUploadFileNum(item.getMinUploadFileNum());
    textInfoItem.setMaxUploadFileNum(item.getMaxUploadFileNum());
    textInfoItem.setType(item.getType());
    textInfoItemList.add(textInfoItem);
}
}
textInfo.setUploadTextInfoItemList(textInfoItemList);
textInfo.setUploadTextExampleItemList(exampleItemList);
return TechnicianResp.success(textInfo);
}

/**
 * 异常报备上传模版
 * @return
 */
@UriMapping(value = "/reportExceptionTemplate")
public Object reportExceptionTemplate() {
    long serviceOrderId = getLongParameter("serviceOrderId");
    try {
        // 校验参数并返回类目id
        String categoryId = checkParamAndReturnCategoryId();
        int techId = getTechId();

        // 查询自检报备
        TechnicianSelfTestRequest request = new TechnicianSelfTestRequest();
        request.setServiceOrderId(String.valueOf(serviceOrderId));
        request.setTechId(techId);
        log.info("reportExceptionTemplate {}", request);
```



```
String version = getStringParameterSafely("version");
List<ExceptionReportQuestionItem> exceptionReportQuestionItemList = null;
if (StringUtils.isEmpty(version) && version.equals(Lion.getString("technician-mobile-web", INSPECTION_VERSION_KEY, "3"))) {
    TechnicianResp<ExceptionReportQuestionItemDTO> resp = doorToDoorTechnicianSelfTestService.querySelfInspectionReportProblems(request);
    if (resp != null && resp.respSuccess() && resp.getData() != null) {// 老老逻辑
        return buildReportExceptionTemplateResponse(resp.getData());
    }
}
// 异常信息为用户不在家时的模板渲染
TechnicianResp customerNotAtHome = handleCustomerNotAtHomeScenario(serviceOrderId, techId);
if (customerNotAtHome != null) return customerNotAtHome;

// 异常报备列表渲染 此时服务人员还没有点击上报异常
if (StringUtils.isEmpty(version) && version.equals("4")) {
    // 查模版信息
    exceptionReportQuestionItemList = LionConfigUtil.queryExceptionReportQuestionItemListNew();
}
// 组装结果
return buildReportExceptionTemplateResponse(categoryId, exceptionReportQuestionItemList);
} catch (Exception e) {
    log.error("reportExceptionTemplate error, serviceOrderId={}", serviceOrderId, e);
    return TechnicianResp.fail("系统繁忙,请稍后再试");
}
}

private TechnicianResp handleCustomerNotAtHomeScenario(long serviceOrderId, int techId){
    DoorToDoorReportExceptionQueryRequest doorToDoorReportExceptionQueryRequest = new DoorToDoorReportExceptionQueryRequest();
    ServiceOrderTechIdPair serviceOrderTechIdPair = new ServiceOrderTechIdPair();
    serviceOrderTechIdPair.setServiceOrderId(serviceOrderId);
    serviceOrderTechIdPair.setTechId(techId);
    List<ServiceOrderTechIdPair> serviceOrderTechIdPairList = Lists.newArrayList();
    serviceOrderTechIdPairList.add(serviceOrderTechIdPair);
    doorToDoorReportExceptionQueryRequest.setServiceOrderTechIdPairList(serviceOrderTechIdPairList);
    TechnicianResp<Boolean> customerNotAtHome = doorToDoorTechnicianReportExceptionHandler.isCustomerNotAtHome(doorToDoorReportExceptionQueryRequest);
    String keyLast = CATEGORY_VALUE + serviceOrderId;
    StoreKey storeKey = new StoreKey(CATEGORY, keyLast);
    Boolean isStart = false;
    String o = null;
    try {
        o = retryer.call(() -> redisStoreClient.get(storeKey));
    } catch (Exception e) {
        log.error("redisStoreClient.get error", e);
    }
    if(o!=null && o.equals("true")){
        isStart=true;
    }
}
```



```
if (customerNotAtHome != null && customerNotAtHome.resSuccess() && customerNotAtHome.getData() != null && !isStart) {
    UploadReportExceptionInfoDTO uploadReportExceptionInfoDTO = new UploadReportExceptionInfoDTO();
    uploadReportExceptionInfoDTO.setPageTitle("请稍等，已短信联系用户");
    uploadReportExceptionInfoDTO.setTextDesc("如需取消订单，可点击下方按钮操作");

    TechnicianResp<ServiceOrderDTO> resp = doorToDoorServiceBcpService.queryServiceOrderInfo(serviceOrderId);
    if(resp.resFail()||resp.getData()==null){
        return TechnicianResp.fail(UploadReportExceptionInfoCodeEnum.Not_Successss.getCode(), "订单预定时间获取失败，请重试");
    }
    ServiceOrderDTO serviceOrderDTO = resp.getData();
    if(serviceOrderDTO.getBookStartTime()==null){
        return TechnicianResp.fail(UploadReportExceptionInfoCodeEnum.Not_Successss.getCode(), "订单预定时间获取失败，请重试");
    }
    Date bookStartTime = serviceOrderDTO.getBookStartTime();
    Date currentTime = new Date();
    // 计算保留时间
    Date reserveTime;
    if (currentTime.before(new Date(bookStartTime.getTime() + 30 * 60 * 1000))) {
        // 当前时间在预约开始时间+30分钟内
        reserveTime = new Date(bookStartTime.getTime() + 30 * 60 * 1000);
    } else {
        // 当前时间在预约开始时间+30分钟后
        reserveTime = new Date(currentTime.getTime() + 10 * 60 * 1000);
    }

    uploadReportExceptionInfoDTO.setReserveTime(reserveTime);

    StoreKey storeKeyTime = new StoreKey(CATEGORY, serviceOrderId);
    Date reserveTimeRes = null;
    try {
        reserveTimeRes = dateRetryer.call(()->redisStoreClient.get(storeKeyTime));
    } catch (Exception e) {
        log.error("reportExceptionTemplate get reserveTimeRes error", e);
    }
    if(reserveTimeRes!=null){
        reserveTime = reserveTimeRes;
    }
    uploadReportExceptionInfoDTO.setReserveTime(reserveTime);

    uploadReportExceptionInfoDTO.setTextDesc("若 {reserveTime} 后仍未与用户取得联系，可点击下方按钮取消订单；若联系上用户可正常开始服务，请点击右下方按钮开始服务");

    List<ReportExceptionButtonDTO> buttonList = Lists.newArrayList();
    ReportExceptionButtonDTO reportExceptionButtonDT01 = new ReportExceptionButtonDTO();
    reportExceptionButtonDT01.setText("取消订单");
    reportExceptionButtonDT01.setType("cancel");
    buttonList.add(reportExceptionButtonDT01);
```



```
ReportExceptionButtonDT0 reportExceptionButtonDT02 = new ReportExceptionButtonDT0();
reportExceptionButtonDT02.setText("开始服务");
reportExceptionButtonDT02.setType("detail");
reportExceptionButtonDT02.setIsRecord(true);
buttonList.add(reportExceptionButtonDT02);
uploadReportExceptionInfoDT0.setButton(buttonList);

uploadReportExceptionInfoDT0.setHasReportedCustomerNotAtHome(true);

uploadReportExceptionInfoDT0.setReportId(SubmitAfterCheckInEnum.CustomerNotAtHome.getCode());

return TechnicianResp.success(uploadReportExceptionInfoDT0);
}
return null;
}

private TechnicianResp<ReportExceptionTemplateResponse> buildReportExceptionTemplateResponse(ExceptionReportQuestionItemDT0 data) {
    ReportExceptionTemplateResponse response = new ReportExceptionTemplateResponse();
    response.setTitle("自检报备");
    buildReportIssueModule(data, response);
    buildEvidencePictureModule(response);
    buildIssueReasonModule(response);
    response.setReportType(ReportTypeEnum.INSPECTION_REPORT.getType());
    log.info("自检报备 :{}", response);
    return TechnicianResp.success(response);
}

private void buildReportIssueModule(ExceptionReportQuestionItemDT0 data, ReportExceptionTemplateResponse response) {
    List<ReportIssueDT0> reportIssueDTOS = data.getReportIssueDTOS();
    ReportIssueModule reportIssueModule = new ReportIssueModule();
    reportIssueModule.setTitle("请选择您需要报备的问题（单选）");
    List<ReportIssue> list = new ArrayList<>();
    for (ReportIssueDT0 reportIssueDT0 : reportIssueDTOS) {
        ReportIssue reportIssue = new ReportIssue();
        BeanUtils.copyProperties(reportIssueDT0, reportIssue);
        List<ReportIssueDT0.SubReportIssueDT0> subReportIssueDTOS = reportIssueDT0.getSubReportIssueDTOS();
        if(CollectionUtils.isNotEmpty(subReportIssueDTOS)){
            reportIssue.setSubReportIssueDTOS(subReportIssueDTOS);
        }
        list.add(reportIssue);
    }
    reportIssueModule.setReportIssueList(list);
    response.setReportIssueModule(reportIssueModule);
}

/**
 * 取消订单
```

```
650 */
651 @UriMapping(value = "/cancelOrder")
652 @Transactional(rollbackFor = Exception.class)
653 public Object cancelOrder() {
654     CancelOrderExceptionRequest request = getRequest(CancelOrderExceptionRequest.class);
655     CancelOrderDTO cancelOrderDTO = new CancelOrderDTO();
656     try {
657         // 校验参数并返回类目id
658         Boolean checkResult = checkParamCancelOrder(request);
659         if(!checkResult){
660             return TechnicianResp.fail("参数错误");
661         }
662         Integer techId = getTechId();
663
664         // 上传其他信息 客户不在家时
665         uploadOrderInfo(request,techId,cancelOrderDTO);
666
667         if(!request.getCancel()){// 不取消订单
668             String keyLast = "status" + request.getServiceOrderId();
669             StoreKey storeKey = new StoreKey(CATEGORY, keyLast);
670             String value = "true";
671             try {
672                 redisStoreClient.set(storeKey, value);
673             } catch (Exception e) {
674                 log.error("reportExceptionTemplate set reserveTimeRes error", e);
675             }
676             return TechnicianResp.success(cancelOrderDTO);
677         }
678
679         // 发送短信
680         Boolean isSend = sendMessageToCustomer(request, CancelAfterSubmitEnum.fromCode(request.getId()), cancelOrderDTO);
681
682         // 取消订单
683         boolean isSuccessCancel = cancelOrderForException(techId, request,cancelOrderDTO);
684         if(!isSuccessCancel){
685             throw new RuntimeException("取消订单失败"); // 抛出异常以回滚事务
686         }
687         // 如果服务超限, 生成判责单
688         if (request.getId()==SubmitAfterCheckInEnum.ServiceAreaExceedsLimits.getCode()){// 服务范围超限
689             createLiabilityOrder(request.getServiceOrderId(),cancelOrderDTO);
690         }
691
692     }catch (Exception e){
693         log.error("系统繁忙,请稍后再试", e);
694         throw new RuntimeException("系统繁忙,请稍后再试"); // 确保事务回滚
695     }
```

杜赞351806768
C-3

return TechnicianResp.success(cancelOrderDT0);

}

private boolean cancelOrderForException(Integer techId, CancelOrderExceptionRequest request, CancelOrderDT0 cancelOrderDT0) {

CancelOrderRequest cancelOrderRequest = new CancelOrderRequest();

TakeOrderRecordDetailDT0 takeOrderRecord = getTakeOrderRecord(techId, request.getServiceOrderId());

if(takeOrderRecord==null){

return false;

}

CancelAfterSubmitEnum cancelAfterSubmitEnum = CancelAfterSubmitEnum.fromCode(request.getId());

if (cancelAfterSubmitEnum == null) {

return false;

}

String reason="劳动者端取消 客户问题-"+cancelAfterSubmitEnum.getDesc();

cancelOrderRequest.setOrderId(takeOrderRecord.getReserveOrderId());

cancelOrderRequest.setOperator(ReserveOperatorEnum.System.type);

cancelOrderRequest.setReason(reason);

try {

ReserveResponse<Boolean> response = reserveOrderProcessService.cancelOrder(cancelOrderRequest);

if(response==null||!response.isSuccess()){

return false;

}

} catch (Exception e) {

log.error("cancelOrderForException error, cancelOrderRequest={},", cancelOrderRequest, e);

return false;

}

cancelOrderDT0.setIsCancelSuccess(true);

return true;

}

private void uploadOrderInfo(CancelOrderExceptionRequest request, Integer techId, CancelOrderDT0 cancelOrderDT0) {

if(request.getId()!=CancelAfterSubmitEnum.CustomerNotAtHome.getCode()){

return;

}

String notCancelMessage="于 %s 正常开始服务";

String cancelMessage="于 %s 确认无法服务并离开";

String time = new SimpleDateFormat("H点mm分").format(new Date());

BatchInsertReportExceptionOtherInfoRequest batchInsertReportExceptionOtherInfoRequest = new BatchInsertReportExceptionOtherInfoRequest();

batchInsertReportExceptionOtherInfoRequest.setServiceOrderId(request.getServiceOrderId());

batchInsertReportExceptionOtherInfoRequest.setTechId(techId);

if(request.getCancel()){

batchInsertReportExceptionOtherInfoRequest.setOtherInfo(String.format(cancelMessage,time));

}else {

batchInsertReportExceptionOtherInfoRequest.setOtherInfo(String.format(notCancelMessage,time));

}

try {

杜赞351806768
C-3

杜赞351806768
C-3

```
742         doorToDoorTechnicianReportExceptionService.batchInsertReportExceptionOtherInfo(batchInsertReportExceptionOtherInfoRequest);
743         cancelOrderDTO.setIsGenerateOrderInfo(true);
744     } catch (Exception e) {
745         log.error("uploadOrderInfo error, request={}", request, e);
746     }
747 }
748
749 private Boolean sendMessageToCustomer(CancelOrderExceptionRequest request, CancelAfterSubmitEnum cancelAfterSubmitEnum, CancelOrderDTO cancelOrderDTO) {
750     TechnicianResp<ServiceOrderDTO> resp = doorToDoorServiceBcpService.queryServiceOrderInfo(request.getServiceOrderId());
751     if (resp.respFail() || resp.getData() == null) {
752         return false;
753     }
754     ServiceOrderDTO serviceOrderDTO = resp.getData();
755     if (serviceOrderDTO.getBookStartTime() == null) {
756         return false;
757     }
758     String mobileNo = getMobileNo(request.getServiceOrderId());
759     Date bookStartTime = serviceOrderDTO.getBookStartTime();
760     log.info("sendMessageToCustomer开始发送短信, reason:{}, templateNO:{}, mobile:{}, request:{},",
761             cancelAfterSubmitEnum.getMessage(), Environment.isOfflineEnv() ? 2015094 : 176502, mobileNo, JsonUtils.toJson(request));
762
763     if (StringUtils.isEmpty(mobileNo)) {
764         String bookStartTimeStr = new SimpleDateFormat("HH:mm").format(bookStartTime);
765         try {
766             SMSRequest smsRequest = new SMSRequest();
767             smsRequest.setMobileNo(mobileNo);
768             smsRequest.setTemplateNO(Environment.isOfflineEnv() ? 2015094 : 176502);
769             Map<String, String> parameter = Maps.newHashMap();
770             parameter.put("reason", cancelAfterSubmitEnum.getMessage());
771             parameter.put("bookStartTime", bookStartTimeStr);
772             smsRequest.setParameter(parameter);
773             technicianSendMessageService.asyncSendSMS(smsRequest);
774             cancelOrderDTO.setIsSendMessage(true);
775             log.info("sendMessageToCustomer短信发送成功, reason:{}, bookStartTime:{}, templateNO:{}, mobile:{}, request:{},",
776                     cancelAfterSubmitEnum.getMessage(), bookStartTimeStr, Environment.isOfflineEnv() ? 2015094 : 176502, mobileNo, JsonUtils.toJson(request));
777         } catch (Exception e) {
778             log.error("sendMessageToCustomer短信发送失败, reason:{}, bookStartTime:{}, templateNO:{}, mobile:{}, request:{},",
779                     cancelAfterSubmitEnum.getMessage(), bookStartTimeStr, Environment.isOfflineEnv() ? 2015094 : 176502, mobileNo, JsonUtils.toJson(request), e);
780         }
781     }
782     return true;
783 }
784
785 private String getMobileNo(Long serviceOrderId) {
786     if (serviceOrderId != null) {
787         // 服务单id查询服务单信息
```



```

788         TechnicianResp<TechServiceOrderDT0> serviceOrderDT0TechnicianResp = doorToDoorTechServiceOrderQueryService.findDetailByOrderId(serviceOrderId);
789         if (serviceOrderDT0TechnicianResp.respFail() || serviceOrderDT0TechnicianResp.getData() == null) {
790             return "";
791         }
792         return serviceOrderDT0TechnicianResp.getData().getUserMobileNo();
793     }
794     return "";
795 }
796
797 private Boolean checkParamCancelOrder(CancelOrderExceptionRequest request) {
798     if(request==null
799         || request.getServiceOrderId()==null
800         || request.getServiceOrderId()<=0
801         || request.getId()==null
802         || request.getId()<=0
803         ||request.getCancel()==null){
804         return false;
805     }
806     boolean flag=false;
807
808     CancelAfterSubmitEnum cancelAfterSubmitEnum = CancelAfterSubmitEnum.fromCode(request.getId());
809     if(cancelAfterSubmitEnum==null){
810         return false;
811     }
812     for(CancelAfterSubmitEnum cancelAfterSubmitEnum1:CancelAfterSubmitEnum.values()){
813         if (cancelAfterSubmitEnum1.getCode()== request.getId()) {
814             flag = true;
815             break;
816         }
817     }
818     if(!flag){
819         return false;
820     }
821     return true;
822 }
823
824 /**
825  * 异常报备信息上传
826  * @return
827  */
828 @UriMapping(value = "/uploadReportExceptionInfo")
829 public Object uploadReportExceptionInfo() {
830     UploadReportExceptionInfoRequest request=getRequest(UploadReportExceptionInfoRequest.class);
831     try {
832         // 校验参数
833         String checkFailText = checkUploadReportExceptionInfoParam(request);

```



```
if (StringUtils.isEmpty(checkFailText)) {
    return TechnicianResp.fail(checkFailText);
}

if(StringUtils.isEmpty(request.getVersion())&&request.getVersion().equals("1")){//新版本
    int techId = getTechId();
    boolean isSignFlag = false;
    boolean isSendMsgFlag = false;
    boolean isCancelFlag = false;
    boolean isContact = false;
    UploadReportExceptionInfoDTO uploadReportExceptionInfoDTO = new UploadReportExceptionInfoDTO();
    uploadReportExceptionInfoDTO.setPageTitle("已报备完成");
    List<ReportExceptionButtonDTO> button = Lists.newArrayList();
    ReportExceptionButtonDTO buttonDTO = new ReportExceptionButtonDTO();
    buttonDTO.setText("返回订单详情页");
    buttonDTO.setType("detail");
    button.add(buttonDTO);
    uploadReportExceptionInfoDTO.setButton(button);
    for (SubmitAfterCheckInEnum submitAfterCheckInEnum:SubmitAfterCheckInEnum.values()){//判断是否需要签到
        if(submitAfterCheckInEnum.getCode()==request.getId()){
            isSignFlag = true;
            break;
        }
    }
    for (SendMessageAfterSubmitEnum sendMessageAfterSubmitEnum:SendMessageAfterSubmitEnum.values()){//判断是否需要触达站长
        if(sendMessageAfterSubmitEnum.getCode()==request.getId()){
            isSendMsgFlag = true;
            break;
        }
    }
    for (CancelAfterSubmitEnum cancelAfterSubmitEnum:CancelAfterSubmitEnum.values()){//判断是否需要取消订单
        if(cancelAfterSubmitEnum.getCode()==request.getId()){
            isCancelFlag = true;
            break;
        }
    }
    for (ContactCustomerEnum contactCustomerEnum : ContactCustomerEnum.values()) {
        if (contactCustomerEnum.getCode() == request.getId()) {
            isContact = true;
            break;
        }
    }
}
if(isContact){
    uploadReportExceptionInfoDTO.setTextDesc("请您主动联系用户说明情况");
    List<ReportExceptionButtonDTO> buttonList = Lists.newArrayList();
    ReportExceptionButtonDTO reportExceptionButtonDTO = new ReportExceptionButtonDTO();
```

```
reportExceptionButtonDT0.setText("联系用户说明情况");
reportExceptionButtonDT0.setType("detail");
buttonList.add(reportExceptionButtonDT0);
uploadReportExceptionInfoDT0.setButton(buttonList);
}
if(isCancelFlag){
    uploadReportExceptionInfoDT0.setTextDesc("如需取消订单, 可点击下方按钮操作");
    List<ReportExceptionButtonDT0> buttonList = Lists.newArrayList();
    ReportExceptionButtonDT0 reportExceptionButtonDT01 = new ReportExceptionButtonDT0();
    reportExceptionButtonDT01.setText("取消订单");
    reportExceptionButtonDT01.setType("cancel");
    buttonList.add(reportExceptionButtonDT01);
    ReportExceptionButtonDT0 reportExceptionButtonDT02 = new ReportExceptionButtonDT0();
    reportExceptionButtonDT02.setText("继续服务");
    reportExceptionButtonDT02.setType("detail");
    reportExceptionButtonDT02.setIsRecord(true);
    buttonList.add(reportExceptionButtonDT02);
    uploadReportExceptionInfoDT0.setButton(buttonList);
}
if(isSignFlag){// 需要签到
    boolean hasSignOrder = doorToDoorTaskCenterService.hasSignOrder(request.getServiceOrderId());
    if(!hasSignOrder){
        uploadReportExceptionInfoDT0.setPageTitle("请先完成签到");
        uploadReportExceptionInfoDT0.setTextDesc("完成签到后, 你可以继续进行异常报备");
        List<ReportExceptionButtonDT0> buttonList = Lists.newArrayList();
        ReportExceptionButtonDT0 reportExceptionButtonDT0 = new ReportExceptionButtonDT0();
        reportExceptionButtonDT0.setText("已到达, 立即签到");
        reportExceptionButtonDT0.setType("detail");
        buttonList.add(reportExceptionButtonDT0);
        uploadReportExceptionInfoDT0.setButton(buttonList);
        return new TechnicianResp(UploadReportExceptionInfoCodeEnum.Not_Check_In.getCode(), "请先完成签到", uploadReportExceptionInfoDT0);
    }
}
if(request.getId()==SubmitAfterCheckInEnum.CustomerNotAtHome.getCode()){// 顾客不在家
    TechnicianResp<ServiceOrderDT0> resp = doorToDoorServiceBcpService.queryServiceOrderInfo(request.getServiceOrderId());
    if(resp.respFail()||resp.getData()==null){
        return TechnicianResp.fail(UploadReportExceptionInfoCodeEnum.Not_Successss.getCode(), "订单预定时间获取失败, 请重试");
    }
    ServiceOrderDT0 serviceOrderDT0 = resp.getData();
    if(serviceOrderDT0.getBookStartTime()==null){
        return TechnicianResp.fail(UploadReportExceptionInfoCodeEnum.Not_Successss.getCode(), "订单预定时间获取失败, 请重试");
    }
    Date bookStartTime = serviceOrderDT0.getBookStartTime();
    Date currentTime = new Date();
    // 计算保留时间
    Date reserveTime;
```



```
if (currentTime.before(new Date(bookStartTime.getTime() + 30 * 60 * 1000))) {
    // 当前时间在预约开始时间+30分钟内
    reserveTime = new Date(bookStartTime.getTime() + 30 * 60 * 1000);
} else {
    // 当前时间在预约开始时间+30分钟后
    reserveTime = new Date(currentTime.getTime() + 10 * 60 * 1000);
}
uploadReportExceptionInfoDTO.setReserveTime(reserveTime);

String key = CATEGORY_VALUE+request.getServiceOrderId();
StoreKey storeKeyTime = new StoreKey(CATEGORY, request.getServiceOrderId());
StoreKey storeKey = new StoreKey(CATEGORY, key);
try {
    setRetryer.call()->{
        redisStoreClient.set(storeKeyTime, reserveTime);
        redisStoreClient.delete(storeKey);
        return null;
    });
} catch (Exception e) {
    log.error("redisStoreClient.set error",e);
}
uploadReportExceptionInfoDTO.setTextDesc("若 {reserveTime} 后联系不上用户，可点击取消订单；若联系上客户，请点击开始服务");
List<ReportExceptionButtonDTO> buttonList = Lists.newArrayList();
ReportExceptionButtonDTO reportExceptionButtonDT01 = new ReportExceptionButtonDTO();
reportExceptionButtonDT01.setText("取消订单");
reportExceptionButtonDT01.setType("cancel");
buttonList.add(reportExceptionButtonDT01);
ReportExceptionButtonDTO reportExceptionButtonDT02 = new ReportExceptionButtonDTO();
reportExceptionButtonDT02.setText("开始服务");
reportExceptionButtonDT02.setType("detail");
reportExceptionButtonDT02.setIsRecord(true);
buttonList.add(reportExceptionButtonDT02);
uploadReportExceptionInfoDTO.setButton(buttonList);

String mobileNo = getMobileNo(request.getServiceOrderId());
log.info("uploadReportExceptionInfo短信开始发送,mobileNo:{},templateNO:{},request:{},mobileNo,
        Environment.isOfflineEnv() ? 2015087 : 176343,JsonUtils.toJson(request));

if (StringUtils.isEmpty(mobileNo)) {
    String reserveTimeStr = new SimpleDateFormat("HH:mm").format(reserveTime);
    String bookStartTimeStr = new SimpleDateFormat("HH:mm").format(bookStartTime);
    try {
        SMSRequest smsRequest = new SMSRequest();
        smsRequest.setMobileNo(mobileNo);
```



```
        smsRequest.setTemplateNO(Environment.isOfflineEnv() ? 2015087 : 176343);
        Map<String, String> parameter = Maps.newHashMap();
        parameter.put("reserveTime", reserveTimeStr);
        parameter.put("bookStartTime", bookStartTimeStr);
        smsRequest.setParameter(parameter);
        technicianSendMessageService.asyncSendSMS(smsRequest);
        uploadReportExceptionInfoDTO.setSmsSendSuccess(true);
        uploadReportExceptionInfoDTO.setPageTitle("请稍等, 已短信联系用户");
        log.info("uploadReportExceptionInfo短信发送成功,mobileNo:{},bookStartTime:{},templateNO:{},request:{})",mobileNo,
                bookStartTimeStr,Environment.isOfflineEnv() ? 2015087 : 176343,JsonUtils.toJson(request));
    } catch (Exception e) {
        log.error("uploadReportExceptionInfo短信发送失败,mobileNo:{},bookStartTime:{},templateNO:{},request:{})",mobileNo,
                bookStartTimeStr,Environment.isOfflineEnv() ? 2015087 : 176343,JsonUtils.toJson(request),e);
    }
}

if(isSendMsgFlag){//需要触达站长
    SendMessageAfterSubmitEnum sendMessageAfterSubmitEnum = SendMessageAfterSubmitEnum.fromCode(request.getId());
    sendMessageForMaster(request,techId, sendMessageAfterSubmitEnum, uploadReportExceptionInfoDTO);
}

// 上传图片/视频/文本
TechnicianResp<Boolean> technicianReportExceptionResp = uploadMediaAndReason(request);
if (technicianReportExceptionResp == null || technicianReportExceptionResp.respFail()) {
    return TechnicianResp.fail(UploadReportExceptionInfoCodeEnum.Not_Successs.getCode(),"异常报备上传失败");
}
return new
TechnicianResp(UploadReportExceptionInfoCodeEnum.Success.getCode(),UploadReportExceptionInfoCodeEnum.Success.getDesc(),uploadReportExceptionInfoDTO);
}

// 上传图片/视频/文本
TechnicianResp<Boolean> technicianReportExceptionResp = uploadMediaAndReason(request);
if (technicianReportExceptionResp == null || technicianReportExceptionResp.respFail()) {
    return TechnicianResp.fail("异常报备上传失败");
}
return TechnicianResp.success(true);
} catch (BizException e) {
    return TechnicianResp.fail(e.getMessage());
} catch (Exception e) {
    log.error("uploadReportExceptionInfo error, request={}", JsonUtils.toJson(request), e);
    return TechnicianResp.fail("系统繁忙,请稍后再试");
}
}

private void sendMessageForMaster(UploadReportExceptionInfoRequest request,Integer techId, SendMessageAfterSubmitEnum sendMessageAfterSubmitEnum,
UploadReportExceptionInfoDTO uploadReportExceptionInfoDTO) {
    if(techId==null||techId<=0||sendMessageAfterSubmitEnum==null||uploadReportExceptionInfoDTO==null){
        return;
    }
}
```

```
1017 }
1018 String masterMis = "";
1019 masterMis = getMasterMis(techId);
1020 TextBody message = new TextBody();
1021 String name = "";
1022 name = doorToDoorTaskCenterService.queryNameByTechId(techId);
1023
1024 String reverseOrderIdAndTime = "";
1025 TechnicianResp<ServiceOrderDT0> resp = doorToDoorServiceBcpService.queryServiceOrderInfo(request.getServiceOrderId());
1026 if(resp.respFail()||resp.getData()==null){
1027     return;
1028 }
1029 ServiceOrderDT0 serviceOrderDT0 = resp.getData();
1030 if(serviceOrderDT0.getBookStartTime()==null){
1031     return;
1032 }
1033 Date bookStartTime = serviceOrderDT0.getBookStartTime();
1034 String bookStartTimeStr = new SimpleDateFormat("MM:dd:HH:mm").format(bookStartTime);
1035 TakeOrderRecordDetailDT0 takeOrderRecord = getTakeOrderRecord(techId, request.getServiceOrderId());
1036 if (takeOrderRecord == null||takeOrderRecord.getReserveOrderId()==null) {
1037     return;
1038 }
1039 reverseOrderIdAndTime= takeOrderRecord.getReserveOrderId() + "(" + bookStartTimeStr +)";
1040
1041 String url = "";
1042 String strFormat = Lion.getString("technician-mobile-web", "technician-mobile-web.haiXing.order.url");
1043 url= String.format(strFormat, takeOrderRecord.getReserveOrderId().toString());
1044
1045 message.setText(String.format(sendMessageAfterSubmitEnum.getMessage(),
1046     name,
1047     reverseOrderIdAndTime,
1048     url));
1049 if(StringUtils.isEmpty(masterMis)){
1050     commonPushBaseService.newBeePush3(message, Collections.singletonList(masterMis), Pusher.LIFE_SERVICE_STARFISH_PLATFORM);
1051     uploadReportExceptionInfoDT0.setSendMasterSuccess(true);
1052 }
1053 }
1054
1055 private String getMasterMis(Integer techId) {
1056     if(techId==null||techId<=0){
1057         return "";
1058     }
1059     String masterMis = "";
1060     SpTechnicianQueryRequest request = new SpTechnicianQueryRequest();
1061     request.setTechnicianId(Long.valueOf(techId));
1062     Response<List<SpTechnicianDT0>> listSpResponse = null;
```

```
1063     try {
1064         listSpResponse = spTechnicianDomainService.queryTechniciansFromDB(request);
1065     } catch (Exception e) {
1066         log.error("getCooperationType-error,techId={}", techId, e);
1067     }
1068     SpTechnicianDT0 spTechnicianDT0 = null;
1069     if (listSpResponse != null && CollectionUtils.isNotEmpty(listSpResponse.getData())) {
1070         spTechnicianDT0 = listSpResponse.getData().get(0);
1071     }
1072     if(spTechnicianDT0 == null){
1073         log.error("查询手艺人合作关系失败, techId is {}", techId);
1074         return "";
1075     }
1076     Integer cooperationType = spTechnicianDT0.getCooperationType();
1077     if(cooperationType==null){
1078         return "";
1079     }
1080     TechnicianAreaRelationDT0 technicianAreaRelationDT0 = null;
1081     if (cooperationType == CooperationTypeEnum.CROWD_SOURCING.getCode()) {
1082         QueryTechnicianAreaRelationReq req = new QueryTechnicianAreaRelationReq();
1083         req.setTechnicianId(Long.valueOf(techId));
1084         Response<List<TechnicianAreaRelationDT0>> listResponse = null;
1085         try {
1086             listResponse = spTechnicianDomainService.queryTechnicianAreaRelation(req);
1087         } catch (Exception e) {
1088             log.error("查询手艺人区域关系失败, techId is {}", techId, e);
1089         }
1090         if (listResponse == null || !listResponse.isSuccess() || CollectionUtils.isEmpty(listResponse.getData())) {
1091             log.error("查询手艺人区域关系失败, techId is {}", techId);
1092             return "";
1093         }
1094         List<TechnicianAreaRelationDT0> data = listResponse.getData();
1095         technicianAreaRelationDT0 = data.get(0);
1096         if (technicianAreaRelationDT0 == null) {
1097             log.error("查询手艺人区域关系失败, techId is {}", techId);
1098             return "";
1099         }
1100         masterMis = technicianAreaRelationDT0.getMis();
1101     }
1102     if(cooperationType==CooperationTypeEnum.SERVICE_PROVIDER.getCode()){
1103         try {
1104             Response<ServiceProviderDT0> serviceProviderDT0Response = serviceProviderQueryService.loadServiceProvider(spTechnicianDT0.getProviderId());
1105             if (serviceProviderDT0Response == null || !serviceProviderDT0Response.isSuccess() || serviceProviderDT0Response.getData() == null) {
1106                 log.error("查询服务商失败, techId is {}", techId);
1107                 return "";
1108             }
1109         }
```

```

1109         masterMis = serviceProviderDTOResponse.getData().getAdminAccountName();
1110     } catch (TException e) {
1111         log.error("查询服务商失败, techId is {}", techId, e);
1112         return "";
1113     }
1114
1115 }
1116 return masterMis;
1117 }
1118
1119 /**
1120  * 发起判责单
1121  * @param serviceOrderId
1122  * @return
1123  */
1124 private void createLiabilityOrder(long serviceOrderId, CancelOrderDTO cancelOrderDTO) {
1125     if (serviceOrderId <= 0) {
1126         return;
1127     }
1128     try {
1129         int techId = getTechId();
1130         TakeOrderRecordDetailDTO takeOrderRecord = getTakeOrderRecord(techId, serviceOrderId);
1131         if (takeOrderRecord == null) {
1132             throw new RuntimeException("获取接单记录失败");
1133         }
1134         Long reserveOrderId = takeOrderRecord.getReserveOrderId();
1135         Integer takeOrderRecordId = takeOrderRecord.getTakeOrderId();
1136         String techName = doorToDoorTaskCenterService.queryNameByTechId(techId);
1137
1138         CreateOrderAndAttrRequest createOrderAndAttrRequest = new CreateOrderAndAttrRequest();
1139         createOrderAndAttrRequest.setReserveOrderId(reserveOrderId);
1140         createOrderAndAttrRequest.setTakeOrderRecordId(Long.valueOf(takeOrderRecordId));
1141         createOrderAndAttrRequest.setTechName(techName);
1142         createOrderAndAttrRequest.setWorkOrderSource(WorkOrderSourceEnum.workerReport.getCode());
1143
1144         Map<String, String> attrs = new HashMap<>();
1145         buildAttrs(attrs, techId);
1146         createOrderAndAttrRequest.setAttrs(attrs);
1147
1148         TechnicianResp<String> result = assessLiabilityOrderProcessService.createOrderAndAttrByReserveOrderId(createOrderAndAttrRequest);
1149         if (result == null || result.respFail() || result.getData() == null) {
1150             log.error("提交工单失败, result is null, techId {}, serviceOrderId {}", techId, serviceOrderId);
1151             throw new RuntimeException("提交工单失败");
1152         }
1153     } catch (Exception e) {
1154         log.error("提交工单失败", e);
1155     }
1156 }

```

```
1155         throw new RuntimeException("提交工单失败");
1156     }
1157     cancelOrderDTO.setIsCancelSuccess(true);
1158 }
1159
1160 private void buildAttrs(Map<String, String> attrs, Integer techId) {
1161     WorkOrderProblemDTO workOrderProblemDTO = new WorkOrderProblemDTO(1, "工单来源", 1, Collections.singletonList((new
1162 WorkOrderProblemChoiceDTO(WorkOrderSourceEnum.workerReport.getCode(), WorkOrderSourceEnum.workerReport.getDesc()))));
1163     List<WorkOrderProblemDTO> problems = new ArrayList<>();
1164     problems.add(new WorkOrderProblemDTO(1, "高危问题", 2, Collections.emptyList()));
1165     problems.add(new WorkOrderProblemDTO(WorkOrderProblemEnum.servicePersonnelDeclinesOrder.getCategoryId(),
1166         WorkOrderProblemEnum.servicePersonnelDeclinesOrder.getCategoryName(),
1167         2,
1168         Collections.singletonList(new WorkOrderProblemChoiceDTO(WorkOrderProblemEnum.servicePersonnelDeclinesOrder.getId(),
1169             WorkOrderProblemEnum.servicePersonnelDeclinesOrder.getDesc()))));
1170     problems.add(new WorkOrderProblemDTO(3, "售后体验问题", 2, Collections.emptyList()));
1171     attrs.put(AssessAttrKeyEnum.WORKORDER_SOURCE.getAttrKey(), JsonUtils.toJson(workOrderProblemDTO));
1172     attrs.put(AssessAttrKeyEnum.WORKORDER_PROBLEM.getAttrKey(), JsonUtils.toJson(problems));
1173     attrs.put(AssessAttrKeyEnum.PROBLEM_DESC.getAttrKey(), "");
1174     attrs.put(AssessAttrKeyEnum.OPERATOR_ID.getAttrKey(), "系统判责");
1175     attrs.put(AssessAttrKeyEnum.TECH_ID.getAttrKey(), String.valueOf(techId));
1176     putStationMasterMis(attrs);
1177 }
1178
1179 private void putStationMasterMis(Map<String, String> attrs) {
1180     if (MapUtils.isEmpty(attrs) || StringUtils.isBlank(attrs.get(AssessAttrKeyEnum.TECH_ID.getAttrKey()))) {
1181         return;
1182     }
1183     String techIdStr = attrs.get(AssessAttrKeyEnum.TECH_ID.getAttrKey());
1184     long techId = Long.parseLong(techIdStr);
1185     QueryTechnicianAreaRelationReq req = new QueryTechnicianAreaRelationReq();
1186     req.setTechnicianId(techId);
1187     Response<List<TechnicianAreaRelationDTO>> listResponse = null;
1188     try {
1189         listResponse = spTechnicianDomainService.queryTechnicianAreaRelation(req);
1190     } catch (Exception e) {
1191         log.error("查询手艺人区域关系失败, techId is {}", techId, e);
1192     }
1193     if (listResponse == null || !listResponse.isSuccess() || org.apache.commons.collections.CollectionUtils.isEmpty(listResponse.getData())) {
1194         log.error("查询手艺人区域关系失败, techId is {}", techId);
1195         return;
1196     }
1197     List<TechnicianAreaRelationDTO> data = listResponse.getData();
1198     TechnicianAreaRelationDTO technicianAreaRelationDTO = data.get(0);
1199     if (technicianAreaRelationDTO == null) {
1200         log.error("查询手艺人区域关系失败, techId is {}", techId);
1201         return;
1202     }
```



```

1247     return recordDetailRes.getData();
1248 }
1249
1250
1251 private String checkUploadReportExceptionInfoParam(UploadReportExceptionInfoRequest request){
1252     if (request == null || request.getServiceOrderId() <= 0 || request.getId() <= 0) {
1253         throw new BizException("入参错误") ;
1254     }
1255     request.setTechId(getTechId());
1256     if (request.getTechId() <= 0) {
1257         throw new BizException("未登陆") ;
1258     }
1259     // 校验服务单和手艺人是否匹配
1260     DoorToDoorTakeOrderRecordDTO doorToDoorTakeOrderRecordDTO = getDoorToDoorTakeOrderRecordDTO(request);
1261     if (doorToDoorTakeOrderRecordDTO == null) {
1262         throw new BizException("接单查询失败") ;
1263     }
1264     // 查类目信息
1265     String categoryId = getCategoryId(request.getServiceOrderId());
1266     if (StringUtils.isEmpty(categoryId)) {
1267         throw new BizException("获取类目失败") ;
1268     }
1269     // 类目是否支持异常报备
1270     if (!LionConfigUtil.checkCategorySupportReportException(categoryId)){
1271         throw new BizException("当前类目不支持异常报备") ;
1272     }
1273     // 当前上传内容是否符合模版要求
1274     if(StringUtils.isNotEmpty(request.getVersion())&&request.getVersion().equals("1")){
1275         return checkTemplateComplianceNew(request, categoryId);
1276     }
1277     return checkTemplateCompliance(request, categoryId);
1278 }
1279
1280 private TechnicianResp<Boolean> uploadMediaAndReason(UploadReportExceptionInfoRequest request) {
1281     TechnicianResp<Boolean> technicianReportExceptionResp = null;
1282     try{
1283         technicianReportExceptionResp = doorToDoorTechnicianReportExceptionService.batchInsertReportException(adaptBatchInsertReportExceptionRequest(request));
1284     }catch (Exception e){
1285         log.error("uploadReportExceptionInfo batchInsertReportException error, request:{})", JsonUtils.toJson(request), e);
1286     }
1287     return technicianReportExceptionResp;
1288 }
1289
1290 private BatchInsertReportExceptionRequest adaptBatchInsertReportExceptionRequest(UploadReportExceptionInfoRequest request) {
1291     BatchInsertReportExceptionRequest batchInsertReportExceptionRequest = new BatchInsertReportExceptionRequest();
1292     batchInsertReportExceptionRequest.setId(request.getId());

```



```
1293 batchInsertReportExceptionRequest.setReason(request.getReason());
1294 batchInsertReportExceptionRequest.setTechId(request.getTechId());
1295 batchInsertReportExceptionRequest.setServiceOrderId(request.getServiceOrderId());
1296 batchInsertReportExceptionRequest.setUserId(getLoginUserIdForMultiPlatform());
1297 batchInsertReportExceptionRequest.setReportTime(new Date().getTime());
1298
1299 List<TechnicianMediaDTO> mediaList = Lists.newArrayList();
1300 List<ReportExceptionMedia> mediaUrlList = request.getMediaUrlList();
1301 if (CollectionUtils.isNotEmpty(mediaUrlList)) {
1302     for (ReportExceptionMedia reportExceptionMedia : mediaUrlList) {
1303         TechnicianMediaDTO technicianMediaDTO = new TechnicianMediaDTO();
1304         technicianMediaDTO.setMediaType(reportExceptionMedia.getMediaType());
1305         technicianMediaDTO.setFilename(reportExceptionMedia.getFilename());
1306         technicianMediaDTO.setMediaUrl(reportExceptionMedia.getMediaUrl());
1307         technicianMediaDTO.setServiceStage(ServiceStageEnum.REPORT_EXCEPTION.getType());
1308         mediaList.add(technicianMediaDTO);
1309     }
1310 }
1311 batchInsertReportExceptionRequest.setMediaUrlList(mediaList);
1312 return batchInsertReportExceptionRequest;
1313 }
1314
1315 /**
1316  * 校验上传的报备信息是否符合要求
1317  * @return
1318  */
1319 private String checkTemplateCompliance(UploadReportExceptionInfoRequest request, String categoryId) {
1320     List<ExceptionReportQuestionItem> exceptionReportQuestionItemList = LionConfigUtil.queryExceptionReportQuestionItemList();
1321
1322     boolean isCheck = false;
1323     for (ExceptionReportQuestionItem item : exceptionReportQuestionItemList) {
1324         if (!String.valueOf(item.getCategoryId()).equals(categoryId)) {
1325             continue;
1326         }
1327         if (item.getId() != request.getId()) {
1328             continue;
1329         }
1330         isCheck = true;
1331         int maxPictureNum = item.getMaxPictureNum();
1332         boolean needPicture = item.isNeedPicture();
1333         boolean needReason = item.isNeedReason();
1334         if (needPicture && CollectionUtils.isEmpty(request.getMediaUrlList())) {
1335             return "请上传图片/视频";
1336         }
1337         if (needPicture && CollectionUtils.size(request.getMediaUrlList()) > maxPictureNum) {
1338             return "图片/视频数量超过限制";
1339         }
1340     }
1341     return null;
1342 }
```

```
1339     }
1340     // 无图片/视频数量限制的问题, 最多5张
1341     if (!needPicture && CollectionUtils.size(request.getMediaUrlList()) > 5){
1342         return "图片/视频数量超过限制";
1343     }
1344     if (needReason && StringUtils.isEmpty(request.getReason())) {
1345         return "请输入具体原因";
1346     }
1347     break;
1348 }
1349 if (!isChecked){
1350     return "报备问题id错误";
1351 }
1352 return null;
1353 }
1354
1355 private String checkTemplateComplianceNew(UploadReportExceptionInfoRequest request, String categoryId) {
1356     List<ExceptionReportQuestionItem> exceptionReportQuestionItemList = LionConfigUtil.queryExceptionReportQuestionItemListNew();
1357
1358     boolean isChecked = false;
1359     for (ExceptionReportQuestionItem item : exceptionReportQuestionItemList) {
1360         List<ExceptionReportQuestionItem> children = item.getChildren();
1361         for (ExceptionReportQuestionItem child : children) {
1362             if (!String.valueOf(child.getCategoryId()).equals(categoryId)){
1363                 continue;
1364             }
1365             if (child.getId() != request.getId()) {
1366                 continue;
1367             }
1368             isChecked = true;
1369             int maxPictureNum = child.getMaxPictureNum();
1370             boolean needPicture = child.isNeedPicture();
1371             boolean needReason = child.isNeedReason();
1372             if (needPicture && CollectionUtils.isEmpty(request.getMediaUrlList())) {
1373                 return "请上传图片/视频";
1374             }
1375             if (needPicture && CollectionUtils.size(request.getMediaUrlList()) > maxPictureNum) {
1376                 return "图片/视频数量超过限制";
1377             }
1378             // 无图片/视频数量限制的问题, 最多5张
1379             if (!needPicture && CollectionUtils.size(request.getMediaUrlList()) > 5){
1380                 return "图片/视频数量超过限制";
1381             }
1382             if (needReason && StringUtils.isEmpty(request.getReason())) {
1383                 return "请输入具体原因";
1384             }
1385         }
1386     }
1387     return isChecked ? "通过" : "不通过";
1388 }
```

```

1385         break;
1386     }
1387 }
1388 if (!isChecked){
1389     return "报备问题id错误";
1390 }
1391 return null;
1392 }
1393
1394 private DoorToDoorTakeOrderRecordDTO getDoorToDoorTakeOrderRecordDTO(UploadReportExceptionInfoRequest request) {
1395     DoorToDoorTakeOrderRecordDTO doorToDoorTakeOrderRecordDTO = null;
1396     try{
1397         TechnicianResp<DoorToDoorTakeOrderRecordDTO> technicianResp = doorToDoorTakeOrderRecordService.queryTakeOrderByTechIdAndOrderId(request.getTechId(),
request.getServiceOrderId());
1398         if (technicianResp != null && technicianResp.respSuccess() && technicianResp.getData() != null) {
1399             doorToDoorTakeOrderRecordDTO = technicianResp.getData();
1400         }
1401     } catch (Exception e) {
1402         log.error("uploadReportExceptionInfo queryTakeOrderByTechIdAndOrderId error, techId={}, serviceOrderId={}", request.getTechId(), request.getServiceOrder
e);
1403     }
1404     return doorToDoorTakeOrderRecordDTO;
1405 }
1406
1407 private TechnicianResp<ReportExceptionTemplateResponse> buildReportExceptionTemplateResponse(String categoryId, List<ExceptionReportQuestionItem>
exceptionReportQuestionItemList) {
1408     ReportExceptionTemplateResponse response = new ReportExceptionTemplateResponse();
1409     response.setTitle("日常报备");
1410     buildReportIssueModule(response, categoryId, exceptionReportQuestionItemList);
1411     buildEvidencePictureModule(response);
1412     buildIssueReasonModule(response);
1413     response.setReportType(ReportTypeEnum.DAILY_REPORTING.getType());
1414     log.info("日常报备 :{}", response);
1415     return TechnicianResp.success(response);
1416 }
1417
1418 private void buildReportIssueModule(ReportExceptionTemplateResponse response, String categoryId, List<ExceptionReportQuestionItem> questionItems) {
1419     ReportIssueModule reportIssueModule = new ReportIssueModule();
1420     reportIssueModule.setTitle("请选择您需要报备的问题（单选）");
1421
1422     // 过滤出当前类目的一级问题
1423     List<ReportIssue> reportIssueList = questionItems.stream()
1424         .filter(item -> String.valueOf(item.getCategoryId()).equals(categoryId))
1425         .map(item -> {
1426             ReportIssue reportIssue = convertToReportIssue(item);
1427             // 处理子问题

```

```
1429         if (CollectionUtils.isNotEmpty(item.getChildren())) {
1430             reportIssue.setHasChildren(true);
1431             reportIssue.setChildren(item.getChildren().stream()
1432                 .map(this::convertToReportIssue)
1433                 .collect(Collectors.toList()));
1434         }
1435         return reportIssue;
1436     })
1437     .collect(Collectors.toList());
1438     reportIssueModule.setReportIssueList(reportIssueList);
1439     response.setReportIssueModule(reportIssueModule);
1440 }
1441
1442 private ReportIssue convertToReportIssue(ExceptionReportQuestionItem item) {
1443     ReportIssue reportIssue = new ReportIssue();
1444     reportIssue.setId(item.getId());
1445     reportIssue.setText(item.getText());
1446     reportIssue.setNeedMedia(item.isNeedPicture());
1447     reportIssue.setNeedReason(item.isNeedReason());
1448     reportIssue.setMaxUploadPicNum(item.getMaxPictureNum());
1449     return reportIssue;
1450 }
1451
1452 private void buildEvidencePictureModule(ReportExceptionTemplateResponse response) {
1453     EvidencePictureModule evidencePictureModule = new EvidencePictureModule();
1454     evidencePictureModule.setTitle("请上传举证图片/视频");
1455     evidencePictureModule.setNote("请拍摄能证明以上问题的图片/视频（1-5张）");
1456     response.setEvidencePictures(evidencePictureModule);
1457 }
1458
1459 private void buildIssueReasonModule(ReportExceptionTemplateResponse response) {
1460     IssueReasonModule issueReasonModule = new IssueReasonModule();
1461     issueReasonModule.setTitle("请描述您的具体情况");
1462     issueReasonModule.setNote("输入具体原因，不超过100字");
1463     issueReasonModule.setMaxNoteSize(100);
1464     response.setIssueReason(issueReasonModule);
1465 }
1466
1467 private String getCategoryId(long serviceOrderId){
1468     if (serviceOrderId <= 0) {
1469         return null;
1470     }
1471     TechnicianResp<TechServiceOrderDTO> serviceOrderResp = null;
1472     try {
1473         serviceOrderResp = doorToDoorTechServiceOrderQueryService.findDetailById(serviceOrderId);
1474     } catch (Exception e) {
```

```
1475         log.error("reportExceptionTemplate query serviceOrder error! serviceOrderId={}, e)", serviceOrderId, e);
1476     }
1477     if (serviceOrderResp == null || serviceOrderResp.getData() == null || StringUtils.isEmpty(serviceOrderResp.getData().getExtraInfo())) {
1478         return null;
1479     }
1480     return DoorToDoorUtils.getServiceOrderCategory(serviceOrderResp.getData());
1481 }
1482
1483 private String checkParamAndReturnCategoryId(){
1484     long serviceOrderId = getLongParameter("serviceOrderId");
1485     if (serviceOrderId <= 0) {
1486         throw new BizException("入参错误") ;
1487     }
1488     // 查类目信息
1489     String categoryId = getCategoryId(serviceOrderId);
1490     if (StringUtils.isEmpty(categoryId)) {
1491         throw new BizException("获取类目失败") ;
1492     }
1493     // 类目是否支持异常报备
1494     if (!LionConfigUtil.checkCategorySupportReportException(categoryId)) {
1495         throw new BizException("当前类目不支持异常报备") ;
1496     }
1497     return categoryId;
1498 }
1499
1500 }
1501 }
```